

Análisis de Series de Tiempo II

Forecasting Secuencial con RNN, LSTM y GRU

MSc. Fernando Silva



Agenda

- Repaso
- Introducción
- Vanilla RNN, BPTT y el problema del gradiente
- LSTM y GRU
- Forecasting Autoregresivo Profundo
- Seq2Seq, Teacher Forcing y Scheduled Sampling
- Regularización recurrente

Repaso

Clase 2

El baseline es el piso, no un trámite

Si tu modelo no le gana a un naive / seasonal naive robusto, no aporta valor.

Tres familias de features temporales

Calendario (estacionalidad), lags (memoria) y ventanas, rolling / expanding / EWM, (nivel, volatilidad, régimen).

Comparar siempre contra el naive

MAE / RMSE / MAPE describen el error; MASE te dice si el modelo realmente aporta sobre el baseline.

El poder no está en el modelo, está en los features

Boosting no ve el tiempo: hay que dárselo. La serie se vuelve tabla con lags + rolling + calendar features.

Toda feature mira solo el pasado

`rolling().shift(1)`, no `rolling()`: el default de pandas filtra el presente.

No hay k universal ni "más es mejor"

Ventana corta = reactiva pero ruidosa, larga = estable pero con lag.
Más features = más señal, pero más overfitting y más NaN.

El DL recién funciona con muchos datos y dependencias largas y no lineales

Clase libre 21 de julio

Clase libre el 14 de julio

Clase 4 seria el 21 de julio (no tomaré en cuenta la asistencia)

Introducción

Introducción

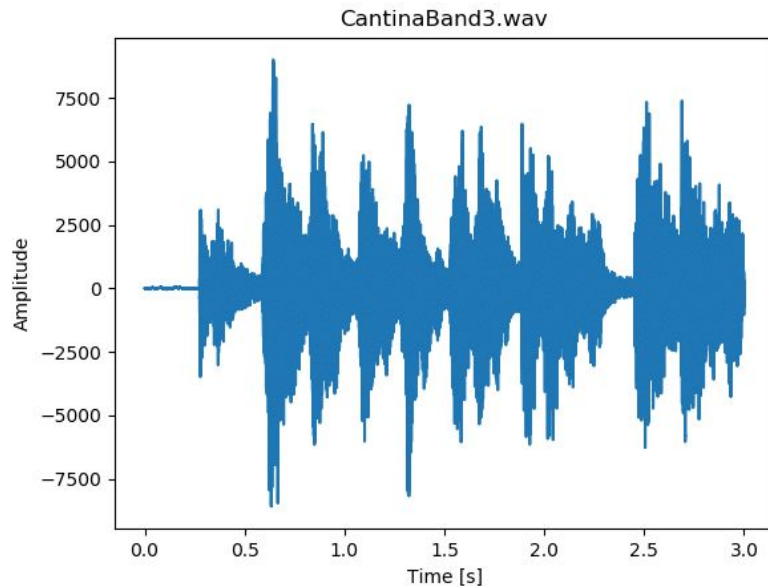
¿Y si el modelo pudiera recordar por sí mismo?

- Clase 1: convertimos el tiempo en un problema supervisado
- Clase 2: construimos la memoria a mano (lags, rolling stats, calendario)
 - Para un modelo tabular, el orden temporal no existe.
 - Todo lo que hemos hecho (ventanas a features a modelo tabular) tiene una limitación estructural: el modelo ve una vista plana del pasado.
 - Para un MLP o un Ridge, "lag_1" y "lag_12" son solo columnas: el orden es una convención, no información.
 - Necesitamos modelos que compartan parámetros a lo largo del tiempo.

Audio

Serie de Tiempo

- Una forma de onda es una serie univariada: amplitud muestreada a f_s Hz
 - $f_s = 11,025 \text{ Hz} \rightarrow 11,025$ observaciones por segundo (los retornos mensuales: 12 por año)
- Aplican los mismos conceptos del curso: ventana, horizonte, split temporal, leakage
- Con audio, el feature engineering de la Clase 2 no es viable: no podemos enumerar a mano 16,000 lags
- Necesitamos modelos que compartan parámetros a lo largo del tiempo



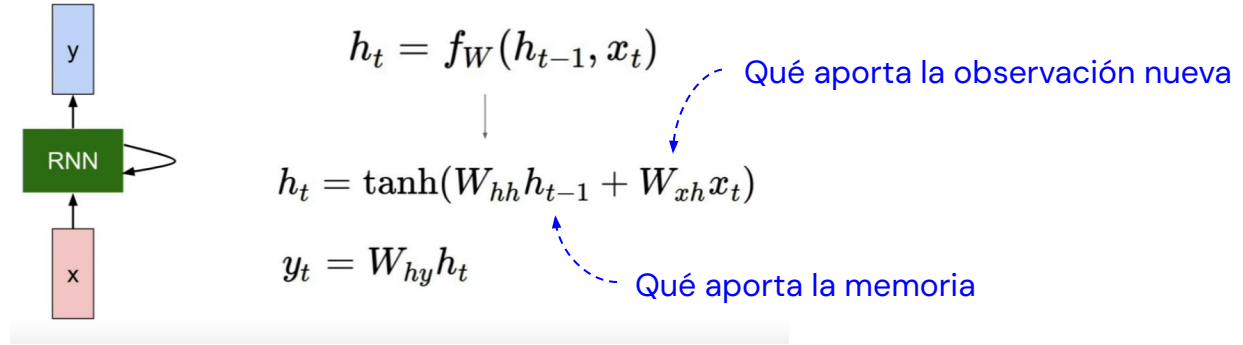
El audio como serie de tiempo

[LINK](#)



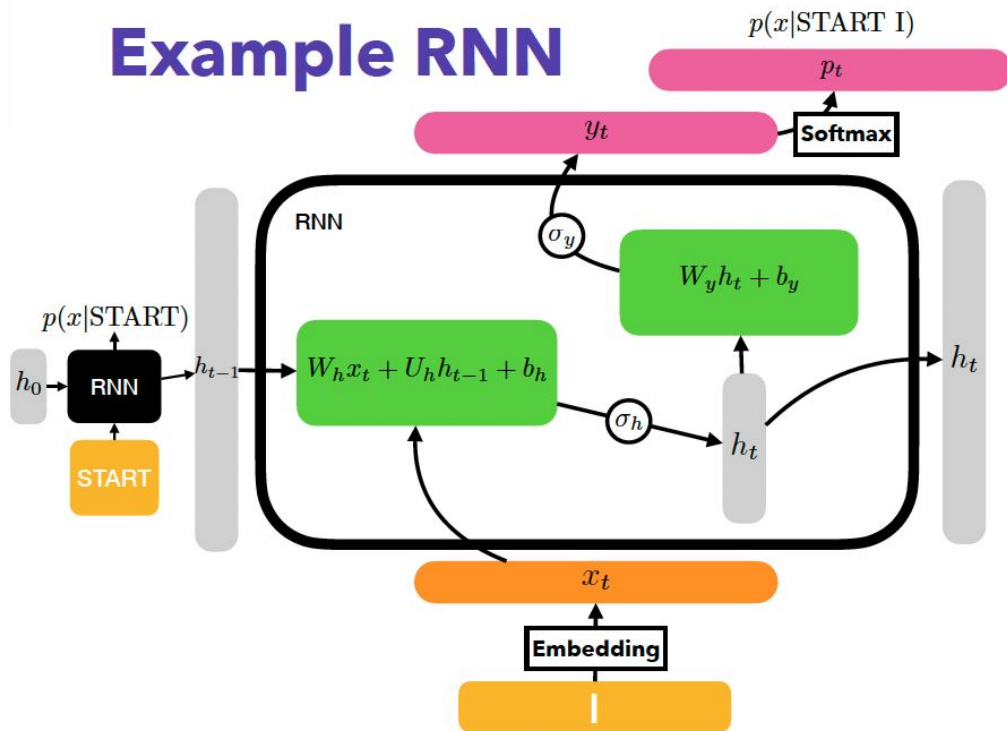
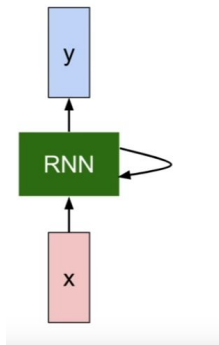
Vanilla RNN, BPTT y el problema del gradiente

Vanilla RNN



- Los mismos pesos se usan en TODOS los pasos (parameter sharing en el tiempo)
- En el MLP cada lag tenía su propio peso, aquí la posición no tiene pesos propios, solo importan el contenido y el orden

Example RNN



Variables:

x_t : input (embedding) vector

y_t : output vector (logits)

p_t : probability over tokens

h_{t-1} : previous hidden vector

h_t : next hidden vector

σ_h : activation function for hidden state

σ_y : output activation function

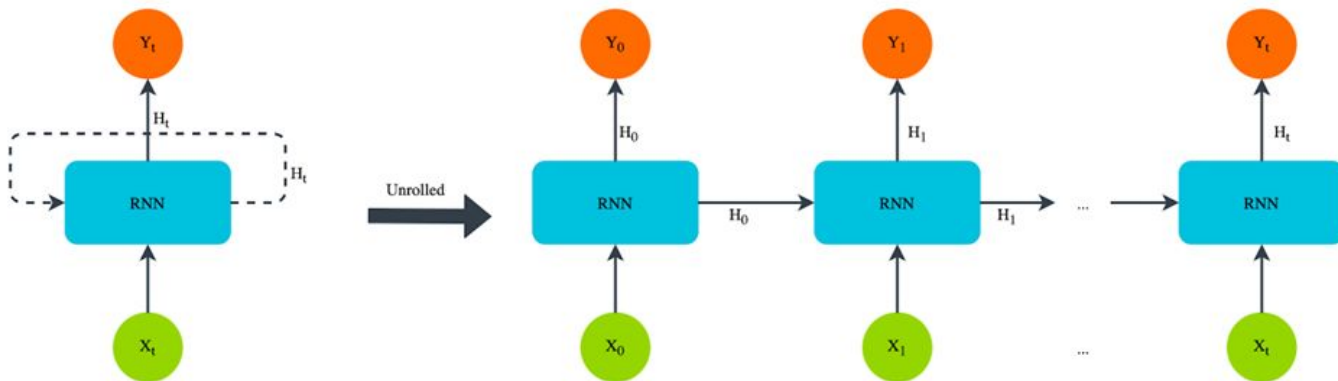
Equations:

$$h_t := \sigma_h(W_h x_t + U_h h_{t-1} + b_h)$$

$$y_t := \sigma_y(W_y h_t + b_y)$$

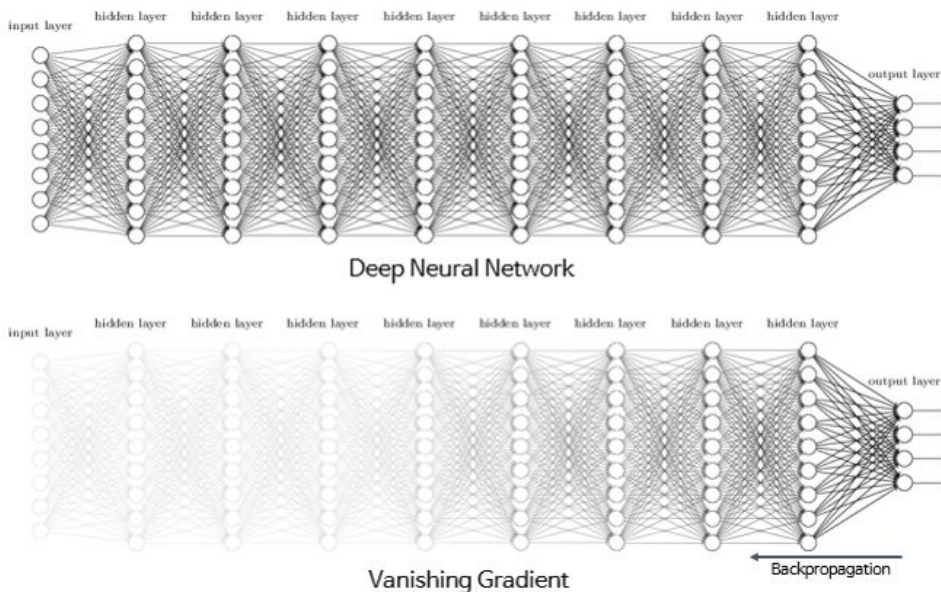
$$p_{t_i} = \frac{\exp(y_{t_i})}{\sum_{i=j}^d \exp(y_{t_j})}$$

Unrolling: la RNN desplegada



- Desplegada en el tiempo, la RNN es una red feedforward profunda donde todas las capas comparten pesos.
- Longitud de la secuencia: una ventana de 100 pasos es como una red de 100 capas.

Problemas de la Vanilla RNN



- BPTT, backprop estándar sobre la red desplegada, regla de la cadena.
- Interés compuesto: $0.9^{100} \approx 0.00003$ (vanishing)... $1.1^{100} \approx 13,780$ (exploding)
- Truncated BPTT: cortar el gradiente cada k pasos (la longitud L del batch ya es un truncamiento implícito)

Limitaciones de la Vanilla RNN

- No aprende dependencias de largo plazo.
 - Tiene dificultades para relacionar información separada por muchos pasos de tiempo.
 - Funciona bien para dependencias cortas, pero olvida información antigua rápidamente.
- Problema del vanishing y exploding gradient.
- No existe un mecanismo para conservar información importante sin modificarla.
- No posee un mecanismo de memoria explícita.
 - Toda la información debe almacenarse en un único estado oculto.
- Bajo rendimiento en secuencias largas.

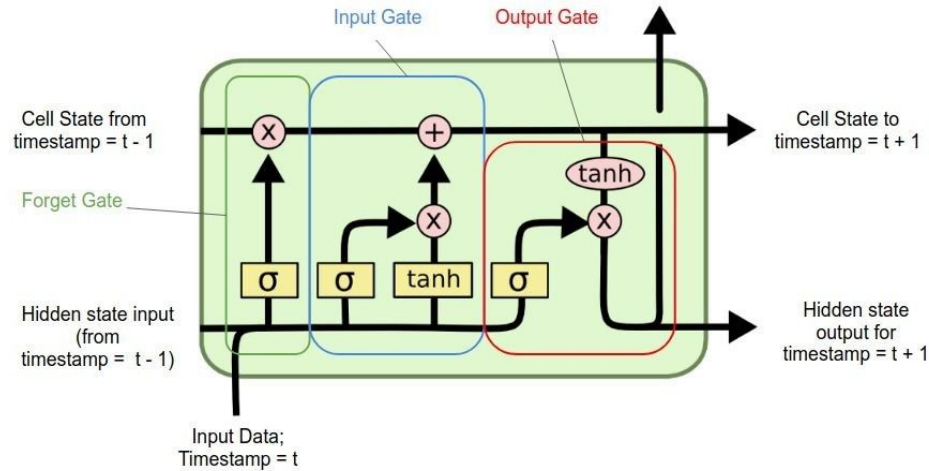
Vanilla RNN y Vanishing Gradient

[LINK](#)



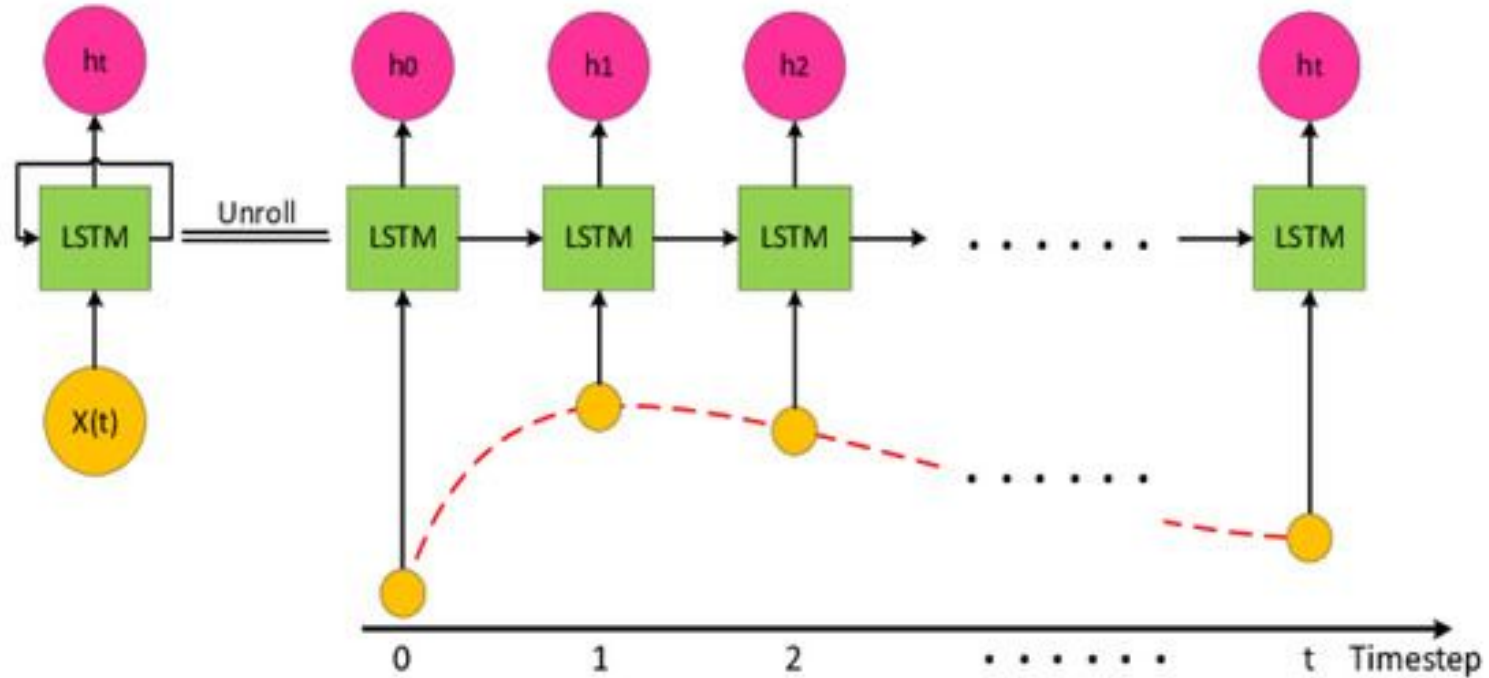
LSTM y GRU

Long Short-Term Memory (LSTM)



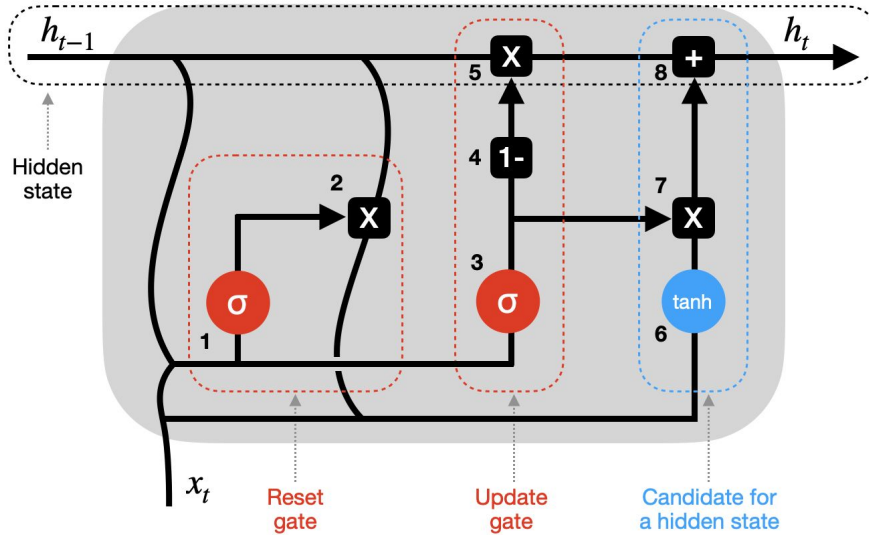
- La RNN vanilla reescribe toda la memoria en cada paso ($h_t = \tanh(\dots)$).
- Solución (LSTM): separar memoria (c_t , cell state) de output (h_t) y controlarla con compuertas.
- Cell state es la cinta transportadora: interacciones aditivas, el gradiente viaja casi intacto.

Long Short-Term Memory (LSTM)



Gated Recurrent Unit (GRU)

GRU Recurrent Unit



h_{t-1} - hidden state at previous timestep t-1 (memory)

x_t - input vector at current timestep t

h_t - hidden state at current timestep t

X - vector pointwise multiplication **+** - vector pointwise addition

tanh - tanh activation function

σ - sigmoid activation function

Y - concatenation of vectors

- states

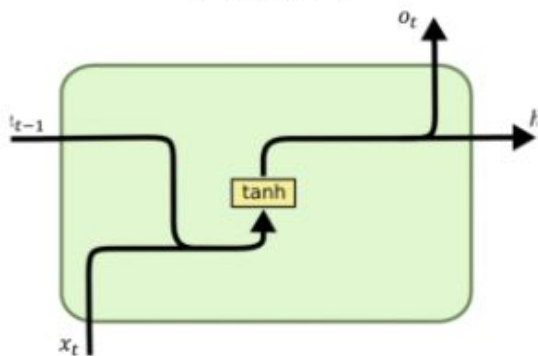
- gates

- updates

- En forecasting LSTM y GRU suelen empatar, GRU es buen default con datos limitados.

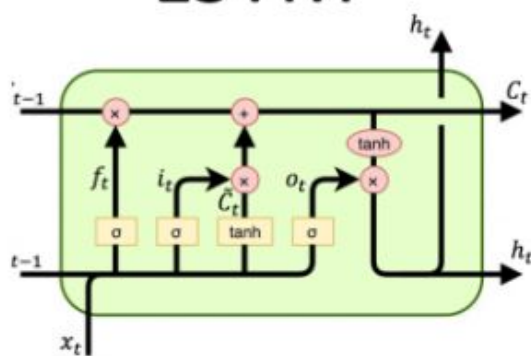
RNN vs LSTM vs GRU

RNN



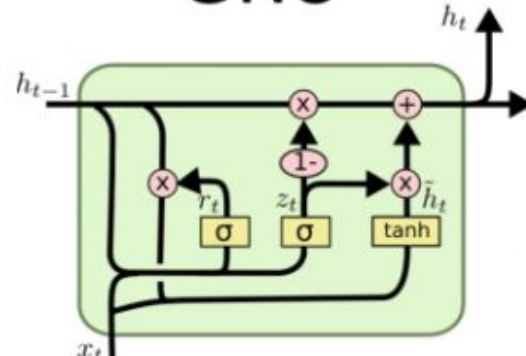
Dependencias cortas y pocos datos

LSTM



LSTM para secuencias largas y más datos.

GRU



GRU como default razonable.

Lo que vimos antes y lo que vemos ahora

Estrategia (Clase 1)	Arquitectura (hoy)	Fortaleza	Costo
Recursive	Rollout autoregresivo (DeepAR-style)	H flexible	Acumula error, exposure bias
Direct	H cabezas sobre h_T	Sin acumulación	Sin coherencia entre horizontes
MIMO	Encoder-Decoder / Seq2Seq	Coherencia conjunta	Requiere teacher forcing

Las estrategias son ortogonales a la arquitectura:
hoy solo les pusimos una red adentro.

LSTM, GRU y Benchmark contra el Naive

[LINK](#)



LSTM, GRU y Benchmark contra el Naive (Caso Apple)

[LINK](#)



Forecasting Autoregresivo Profundo

Rollout recursivo con RNN

- Alimentar la ventana, obtener $\hat{y}_{T+1}$; realimentar $\hat{y}_{T+1}$ como observación, obtener $\hat{y}_{T+2}$; repetir H veces
- El hidden state se arrastra: no se reprocesa toda la ventana en cada paso
- En entrenamiento la red solo vio datos reales; en inferencia se alimenta de sus propias predicciones. Esa discrepancia tiene nombre, exposure bias, y nos la encontraremos de frente en teacher forcing

Acumulación de Error

Audio

- El recursive profundo hereda el problema del recursive clásico: los errores se re-ingieren.
- El error del recursive crece con el horizonte más rápido que el del direct.
- El agravante nuevo es el exposure bias: el modelo nunca vio sus propias predicciones en entrenamiento

Rollout Recursivo

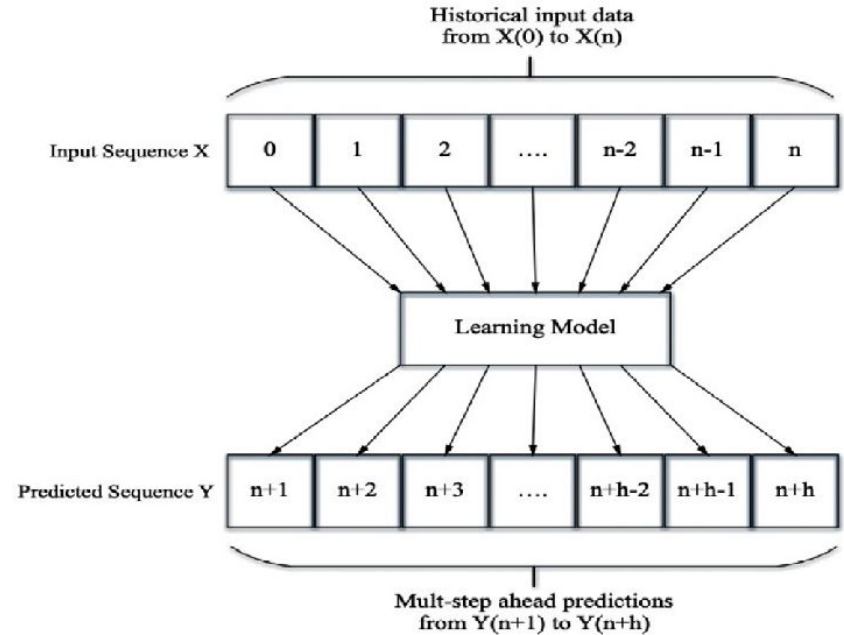
[LINK](#)



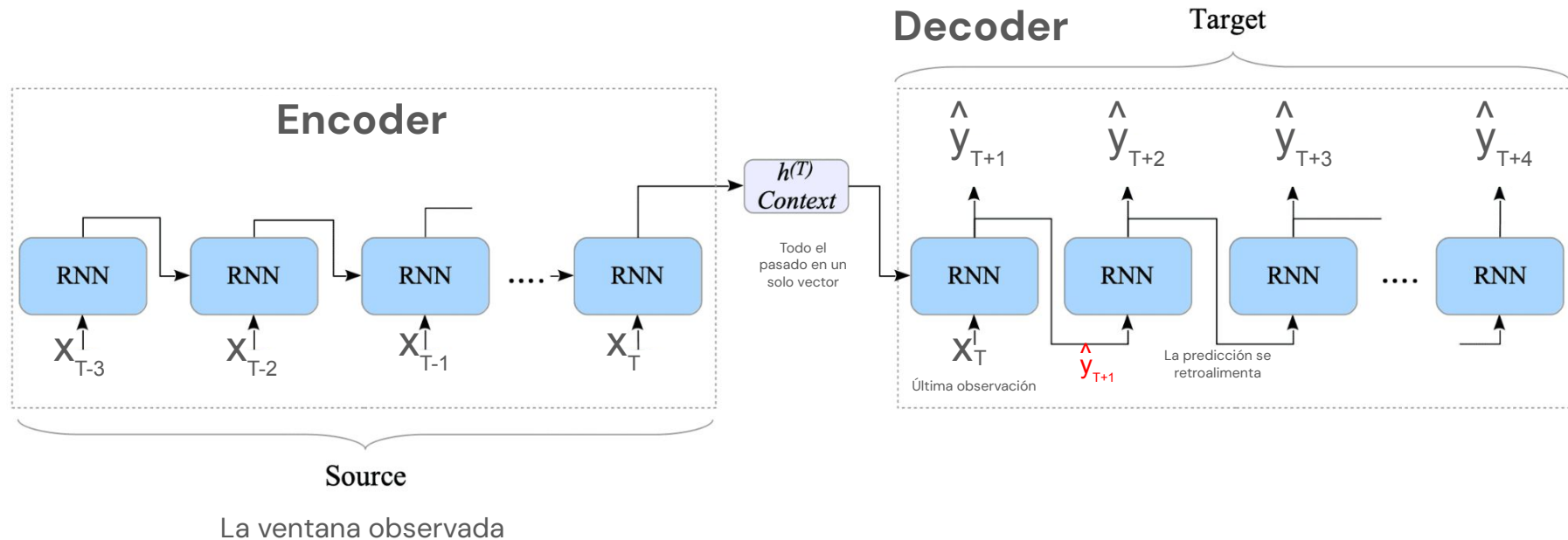
Seq2Seq,
Teacher Forcing y Scheduled Sampling

MIMO profundo es Seq2Seq

- MIMO es un modelo, H salidas simultáneas, coherencia entre horizontes.
- Con redes recurrentes, MIMO se llama Encoder-Decoder / Seq2Seq
- El decoder recurrente va más lejos: cada predicción condiciona la siguiente a través del estado, modela la dependencia conjunta entre horizontes



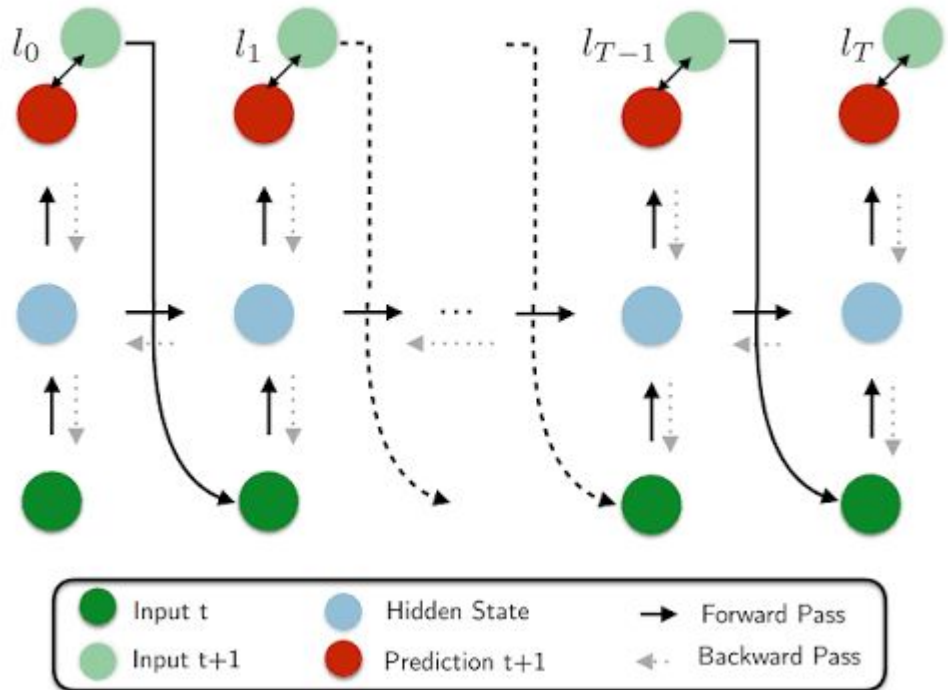
Encoder–Decoder Temporal



Teacher Forcing

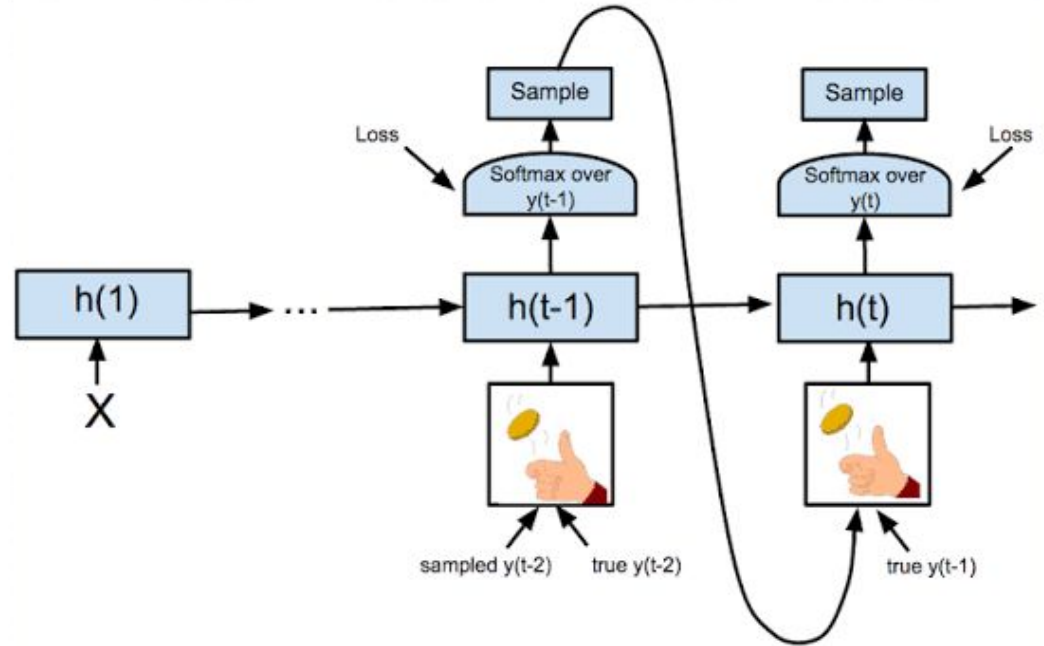
- Teacher forcing es darle el valor real durante el entrenamiento
- Entrenamiento estable y rápido: los errores tempranos no contaminan los pasos siguientes.
- En inferencia no hay valores reales: el decoder se alimenta de sí mismo y pisa terreno que nunca vio en entrenamiento

RNN Training with Teacher Forcing



Scheduled sampling

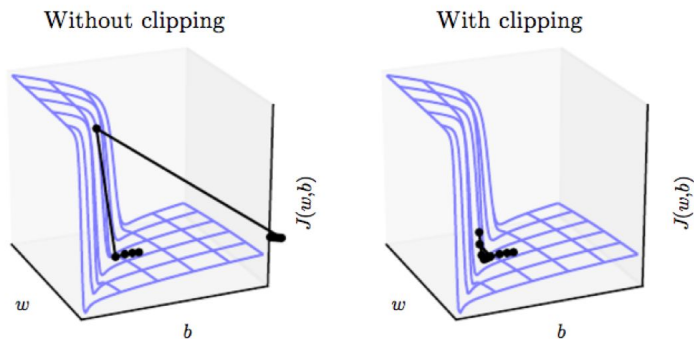
- Durante el entrenamiento: input real con probabilidad p , predicción propia con probabilidad $1-p$
- p decae con las épocas: empieza ~ 1 (puro teacher forcing) y termina cerca de 0 (condiciones de inferencia)
- Esquemas de decaimiento: lineal · exponencial · inverse sigmoid
- Es un curriculum: primero con instructor, luego soltando gradualmente



Regularización recurrente

Regularización Recurrente

a) Gradient Clipping



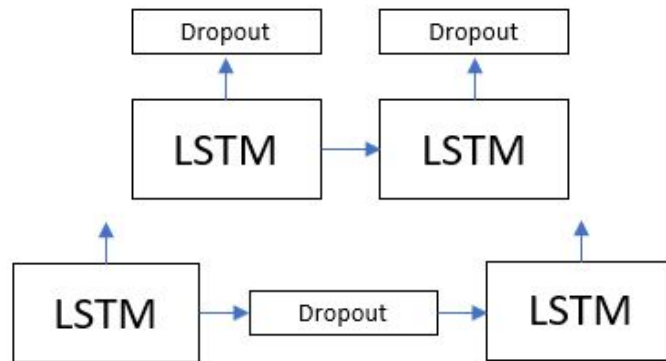
Standard gradient $g(\theta) \doteq \frac{1}{N} \sum_{n=1}^N \nabla \ell_{\theta}(x_n, y_n)$

Clipped gradient $\bar{g}_{\tau}(\theta) \doteq \text{clip}_{\tau}(g(\theta))$

Clip norm to τ

max_norm ≈ 1.0 como default

b) Dropout



Una máscara de dropout distinta por paso temporal rompe la memoria (borra recuerdos al azar en cada instante)

Dropout solo entre capas o variational dropout: la misma máscara en todos los pasos

c) Weight decay y early stopping

d) Reducir capacidad